Lab 5: Static Code Analysis Report

Name: Shravani Prasad Joshi

SRN:PES1UG24AM264

Repository: https://github.com/shravaniprasadjoshi-hub/AM264_Lab5

Aim

The aim of this lab was to run Pylint, Bandit and Flake8 on an assigned Python script, fix the problems they report, and compare the reports before and after the changes. My assigned script was tower_maintenance.py from the repository L5_25_tower_maintenance.

Tools and Environment

I worked on Windows using Git Bash and VS Code, with Python 3.13.6 inside a virtual environment created with python -m venv venv. The three tools were installed with pip.

Tool	Version	What it checks
Pylint	4.1.3	Code quality, naming, docstrings, design problems
Bandit	1.9.4	Security problems and unsafe coding patterns
Flake8	7.4.1	PEP 8 style, spacing and line length

Program Description

tower_maintenance.py is a small program that keeps records of mobile towers in an in-memory SQLite database. Each tower has a site name, the year it was last serviced and a fault count.

The script has functions to add a tower, mark a tower as serviced, list towers that have not been serviced for two or more years, classify a tower as high or normal risk based on its faults, and return all records. The main function adds two towers, services one of them and prints the results. Before and after the changes, the program printed the same output: ['Ridge-A'] high 2 1.

Issues Identified and Fixes

The three tools reported 29 problems in total on the original file. The table lists them by tool, using the line numbers of the original script.

Tool	Issue	Line(s)	Description	Fix
Bandit	SQL injection (B608, Medium)	11	The INSERT query was built with % string formatting	Used a parameterized query with ? placeholders
Bandit	Unsafe pickle (B301 Medium, B403 Low)	1, 43	pickle.loads was used on data that had just been created by pickle.dumps	Removed the pickle round trip and the import, and returned the query result directly
Bandit	Hardcoded password (B105,	6	NOC_PASSWORD was written directly in the source	Read the value from the NOC_PASSWORD

Tool	Issue	Line(s)	Description	Fix
	Low)			environment variable
Pylint	Broad exception (W0718)	20	except Exception hides unrelated errors	Caught sqlite3.Error instead
Pylint	Global statement (W0603, C0103)	7, 16	A global counter named in lower case was changed inside a function	Replaced it with a module level dictionary SERVICE_STATS
Pylint	Naming (C0103)	10, 15, 34	addTower, serviceTower and riskLevel are not snake_case	Renamed to add_tower, service_tower and risk_level
Pylint	Missing docstrings (C0114, C0116)	1, 10, 15, 25, 34, 42, 46	The module and every function had no docstring	Added a docstring to the module and to each function
Pylint	Unnecessary else (R1705)	36	else after a return statement	Removed the else and returned directly
Pylint	Formatting style (C0209)	11	The string could be an f-string	Resolved by the parameterized query
Pylint	Line too long (C0301)	5, 11, 18	Lines longer than 100 characters	Split the lines
Flake8	Line too long (E501)	5, 11, 18, 35, 43	Lines longer than 79 characters	Wrapped the SQL strings and calls over several lines
Flake8	Too many blank lines (E303)	25	Three blank lines before a function	Reduced to two
Flake8	Missing whitespace (E225)	50	total=len(snapshot()) had no spaces around =	Changed to total = len(snapshot())
Flake8	Whitespace at the end of lines and file (W291, W293, W391)	21, 72, 73	Extra spaces and blank lines appeared while I was editing	Removed them

Results Before and After

After the fixes, Pylint rated the code 10.00/10 and Bandit and Flake8 reported no issues.

Measure	Before	After
Pylint score	5.14/10	10.00/10
Pylint messages	18	0
Bandit issues	4 (2 Medium, 2 Low)	0

Measure	Before	After
Flake8 issues	7	0

The before and after report files are stored in the reports folder of the repository. The Flake8 report after the fixes is empty because Flake8 prints nothing when it finds no violations.

Commits

The fixes were made in three commits, one for each tool, after a baseline commit that holds the original reports.

1. Add baseline static analysis reports: the Pylint, Bandit and Flake8 reports from the original script.
2. Fix Bandit issues: parameterized SQL, remove pickle, env password.
3. Fix Pylint issues: docstrings, snake_case, specific exception, no global.
4. Fix Flake8 style issues: line length, blank lines and spacing, plus the final reports in the reports folder.

The commit history is in the repository linked at the top of this report.

Reflection

Which issues were easiest and hardest to fix?

The Flake8 problems and the Pylint naming and docstring warnings were the easiest, since the reports told me exactly what to change. The hardest were the global counter and the hardcoded password, because I had to change how the data was stored or read without changing what the program prints.

Did the tools report any false positives?

I did not find a clear false positive. The pickle warning was correct, but the better fix was to remove the pickle call completely, since it only copied the data. The password warning was also correct even though the value was never used in the program, because passwords should not be stored in source code. The tools also disagreed on one default: Pylint allows lines up to 100 characters and Flake8 allows 79, so a line could pass one and fail the other.

How would I use static analysis in a real project?

I would run all three tools before each commit and in a CI pipeline, so that style and security problems are caught before the code is merged. Bandit is the one I would not skip, because it found a SQL injection problem that I would not have noticed by reading the code.

What improvement did I see after the fixes?

The Pylint score went from 5.14 to 10.00, the four Bandit findings were removed and Flake8 had nothing left to report. The program still gave the same output, and the code is easier to read because of the docstrings and consistent names.